

Les méthodes d'ensemble

Bagging, Random Forest, Boosting, AdaBoost,
Gradient Boosting et XGBoost — avec démonstrations détaillées

5 juin 2026

Table des matières

1	Cadre général	1
1.1	Deux grandes familles, lues à travers le biais et la variance	1
2	Bagging (Bootstrap Aggregating)	2
2.1	Le bootstrap, en détail	2
2.2	Définition et agrégation	2
2.3	Réduction de variance (démonstration)	2
2.4	Pourquoi ça marche surtout pour les arbres	3
3	Random Forest	3
3.1	Motivation : décorréler les arbres	3
3.2	Principe	3
3.3	Interprétation biais-variance	3
4	Boosting : le modèle additif	3
5	AdaBoost	4
5.1	Cadre	4
5.2	La perte exponentielle	4
5.3	Optimisation « stagewise » : dérivation du poids α_m	4
5.4	Mise à jour des poids	5
5.5	Interprétation biais-variance	5
6	Gradient Boosting	5
6.1	Motivation	5
6.2	Descente de gradient dans l'espace des fonctions	5
6.3	Rôle des arbres et mise à jour	6
6.4	Lien avec AdaBoost	6
6.5	Interprétation biais-variance	6
7	XGBoost	6
7.1	Objectif régularisé	6
7.2	Approximation de Taylor d'ordre 2	6
7.3	Poids optimal d'une feuille (dérivation)	7
7.4	Gain d'une coupure (<i>split</i>)	7
7.5	Interprétation biais-variance	7
8	Synthèse biais-variance	8

1 Cadre général

Soit un jeu de données supervisé

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \quad \mathbf{x}_i \in \mathbb{R}^p, y_i \in \mathcal{Y}.$$

On cherche à approximer la fonction cible (régression ou décision)

$$f^*(\mathbf{x}) = \mathbb{E}[Y | X = \mathbf{x}]$$

par un estimateur $\hat{f}(\mathbf{x})$.

L'idée d'ensemble

Plutôt que d'apprendre *un seul* modèle complexe, on apprend **plusieurs modèles simples** (les *apprenants faibles*, *weak learners*), puis on les combine :

$$\hat{f}(\mathbf{x}) = \sum_{m=1}^M \alpha_m h_m(\mathbf{x}).$$

Chaque modèle faible h_m capture une partie de la structure : une région de l'espace, un motif simple, ou l'erreur laissée par les précédents.

Apprenant faible

Un **apprenant faible** est un modèle à peine meilleur que le hasard (p. ex. un arbre de décision très peu profond, un « souche » / *stump*). L'art des méthodes d'ensemble est de transformer une foule d'apprenants médiocres en un prédicteur fort.

1.1 Deux grandes familles, lues à travers le biais et la variance

Rappelons la décomposition de l'erreur : erreur = biais² + variance + bruit.

- **Bagging / Random Forest** (parallèle) : on entraîne des modèles *indépendants* et on les moyenne. Cible principale : **réduire la variance**.
- **Boosting** (séquentiel) : chaque modèle corrige les erreurs du précédent. Cible principale : **réduire le biais**.

2 Bagging (Bootstrap Aggregating)

2.1 Le bootstrap, en détail

Définition 2.1 (Échantillon bootstrap). Un **échantillon bootstrap** $\mathcal{D}^{(b)}$ est obtenu en tirant n exemples *avec remise* dans $\mathcal{D}$ (qui en contient n). Certains exemples apparaissent plusieurs fois, d'autres sont absents.

La règle des 63,2 %

Quelle est la probabilité qu'un exemple donné *ne soit jamais* tiré en n tirages ? À chaque tirage, il a une chance $1 - \frac{1}{n}$ d'être évité, donc

$$\left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} \frac{1}{e} \approx 0,368.$$

Ainsi, chaque échantillon bootstrap contient en moyenne $\approx 63,2\%$ d'exemples *distincts*. Les $\approx 36,8\%$ laissés de côté forment l'ensemble **out-of-bag** (OOB), qui sert de jeu de validation gratuit.

2.2 Définition et agrégation

On génère B échantillons bootstrap et on entraîne un modèle *indépendamment* sur chacun :

$$\mathcal{D}^{(b)} \sim \text{Bootstrap}(\mathcal{D}), \quad \hat{f}_b(\mathbf{x}) = \hat{f}(\mathbf{x}; \mathcal{D}^{(b)}), \quad b = 1, \dots, B.$$

L'agrégation se fait par moyenne (régression) ou vote majoritaire (classification) :

$$\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x}).$$

2.3 Réduction de variance (démonstration)

Supposons que chaque modèle a la même espérance et la même variance, et que deux modèles distincts ont une corrélation constante ρ :

$$\mathbb{E}[\hat{f}_b(\mathbf{x})] = \mu, \quad \text{Var}(\hat{f}_b(\mathbf{x})) = \sigma^2, \quad \text{Corr}(\hat{f}_b, \hat{f}_{b'}) = \rho \quad (b \neq b').$$

On calcule la variance de la moyenne. On développe la variance d'une somme (variances + covariances) :

$$\begin{aligned} \text{Var}(\hat{f}_{\text{bag}}) &= \frac{1}{B^2} \text{Var}\left(\sum_{b=1}^B \hat{f}_b\right) = \frac{1}{B^2} \left[\underbrace{\sum_b \text{Var}(\hat{f}_b)}_{B \text{ termes} = B\sigma^2} + \underbrace{\sum_{b \neq b'} \text{Cov}(\hat{f}_b, \hat{f}_{b'})}_{B(B-1) \text{ termes} = B(B-1)\rho\sigma^2} \right] \\ &= \frac{1}{B^2} (B\sigma^2 + B(B-1)\rho\sigma^2) = \boxed{\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2}. \end{aligned}$$

(On a utilisé $\text{Cov}(\hat{f}_b, \hat{f}_{b'}) = \rho\sigma^2$.) Quand $B \rightarrow \infty$, le second terme disparaît :

$$\text{Var}(\hat{f}_{\text{bag}}) \xrightarrow{B \rightarrow \infty} \rho\sigma^2.$$

À retenir

Moyenner annule la partie « aléatoire » de la variance ($\frac{1-\rho}{B}\sigma^2$), mais il reste un **plancher** $\rho\sigma^2$ dû à la corrélation entre modèles. Pour descendre plus bas, il faut **décorrélérer** les modèles (réduire ρ) : c'est exactement l'idée du Random Forest.

2.4 Pourquoi ça marche surtout pour les arbres

Le bagging agit sur la variance. Les arbres de décision profonds sont des modèles **instables** : une petite variation des données produit un arbre très différent (variance élevée, biais faible). Ce sont donc des candidats idéaux : moyenner beaucoup d'arbres bruités lisse les fluctuations *sans* augmenter le biais.

3 Random Forest

3.1 Motivation : décorrélérer les arbres

Le bagging seul plafonne à $\rho\sigma^2$. Si tous les arbres utilisent les mêmes variables fortes, ils se ressemblent (ρ grand). Le **Random Forest** ajoute une source d'aléa pour les rendre différents.

3.2 Principe

Chaque arbre $\hat{f}_b$ est appris sur :

- un échantillon **bootstrap** $\mathcal{D}^{(b)}$ (comme le bagging) ;
- **et**, à *chaque nœud*, une **sélection aléatoire** d'un sous-ensemble de $m < p$ variables candidates pour le *split*.

Le prédicteur final est la moyenne (ou le vote) :

$$\hat{f}_{\text{RF}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x}).$$

Le rôle de m

En restreignant les variables candidates à chaque nœud, on empêche tous les arbres de se ruer sur la même variable dominante. Les arbres explorent des structures différentes $\Rightarrow \rho$ diminue $\Rightarrow$ le plancher $\rho\sigma^2$ baisse. Choix usuels : $m \approx \sqrt{p}$ (classification), $m \approx p/3$ (régression). C'est le paramètre clé : trop petit, le biais monte ; trop grand, les arbres se recroisent.

3.3 Interprétation biais–variance

Le Random Forest agit principalement sur la **variance** (moyenne + décorrélation). Le biais peut être légèrement supérieur à celui d'un arbre unique, mais la forte réduction de variance domine : meilleure généralisation. Bonus pratique : l'erreur **OOB** donne une estimation de la performance sans jeu de validation séparé, et on obtient gratuitement une mesure d'**importance des variables**.

4 Boosting : le modèle additif

Le boosting construit un modèle **additif**, terme par terme :

$$F_M(\mathbf{x}) = \sum_{m=1}^M \nu \alpha_m h_m(\mathbf{x}),$$

où h_m est l'apprenant faible de l'itération m , α_m son poids, et $\nu \in (0, 1]$ le **taux d'apprentissage** (*shrinkage*).

Modélisation additive progressive (*forward stagewise*)

Contrairement au bagging (parallèle), le boosting est **séquentiel** : on ne revient pas sur les termes déjà placés. À chaque étape on ajoute un terme qui corrige le modèle courant :

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu h_m(\mathbf{x}).$$

Chaque itération se déplace dans la direction qui réduit le plus la perte : c'est une descente *dans l'espace des fonctions*.

À quoi sert ν ?

Le shrinkage ν « freine » chaque pas. Un ν petit (p. ex. 0,1) demande plus d'arbres mais généralise mieux : on avance à petits pas prudents plutôt qu'à grandes enjambées qui sur-apprennent.

5 AdaBoost

5.1 Cadre

AdaBoost combine des classifieurs très simples, à peine meilleurs que le hasard, en mettant l'accent sur les observations **difficiles**. On note les étiquettes et prédictions dans $\{-1, +1\}$:

$$y_i \in \{-1, +1\}, \quad h_m(\mathbf{x}) \in \{-1, +1\}, \quad F_M(\mathbf{x}) = \sum_{m=1}^M \alpha_m h_m(\mathbf{x}), \quad \hat{y} = \text{sign}(F_M(\mathbf{x})).$$

Chaque h_m est entraîné avec une distribution de **poids** $w_i^{(m)}$ sur les exemples.

5.2 La perte exponentielle

AdaBoost minimise la **perte exponentielle** :

$$L(F) = \sum_{i=1}^n \exp(-y_i F(\mathbf{x}_i)).$$

Intuition

La marge $y_i F(\mathbf{x}_i)$ est positive si la prédiction est bonne, négative sinon. L'exponentielle pénalise *très* fortement les marges négatives : un exemple mal classé pèse énormément, d'où la concentration sur les cas difficiles.

5.3 Optimisation « stagewise » : dérivation du poids α_m

À l'itération m , on ajoute αh_m à F_{m-1} . La perte devient

$$L = \sum_{i=1}^n \exp(-y_i (F_{m-1}(\mathbf{x}_i) + \alpha h_m(\mathbf{x}_i))) = \sum_{i=1}^n \underbrace{\exp(-y_i F_{m-1}(\mathbf{x}_i))}_{w_i^{(m)}} \exp(-\alpha y_i h_m(\mathbf{x}_i)).$$

On voit apparaître naturellement les poids $w_i^{(m)} = \exp(-y_i F_{m-1}(\mathbf{x}_i))$. Comme $y_i, h_m \in \{-1, +1\}$, le produit $y_i h_m(\mathbf{x}_i)$ vaut $+1$ si l'exemple est **bien classé** et -1 s'il est **mal classé**. On sépare la somme :

$$L = e^{-\alpha} \sum_{i: \text{correct}} w_i^{(m)} + e^{+\alpha} \sum_{i: \text{erreur}} w_i^{(m)}.$$

En notant l'**erreur pondérée** $\varepsilon_m = \frac{\sum_{i: \text{erreur}} w_i^{(m)}}{\sum_i w_i^{(m)}} = \sum_{i=1}^n w_i^{(m)} \mathbb{1}(y_i \neq h_m(\mathbf{x}_i))$ (poids normalisés), minimiser L revient à minimiser

$$g(\alpha) = (1 - \varepsilon_m) e^{-\alpha} + \varepsilon_m e^{\alpha}.$$

On annule la dérivée :

$$\frac{dg}{d\alpha} = -(1 - \varepsilon_m) e^{-\alpha} + \varepsilon_m e^{\alpha} = 0 \implies \varepsilon_m e^{\alpha} = (1 - \varepsilon_m) e^{-\alpha} \implies e^{2\alpha} = \frac{1 - \varepsilon_m}{\varepsilon_m}.$$

D'où le poids optimal de l'apprenant m :

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_m}{\varepsilon_m} \right) \quad (1)$$

Intuition

Si $\varepsilon_m < 0,5$ (mieux que le hasard), alors $\alpha_m > 0$: l'apprenant compte positivement. Plus il est précis ($\varepsilon_m \rightarrow 0$), plus $\alpha_m \rightarrow +\infty$: on lui fait grande confiance. À $\varepsilon_m = 0,5$, $\alpha_m = 0$: un apprenant au niveau du hasard est ignoré.

5.4 Mise à jour des poids

Par définition, $w_i^{(m+1)} = \exp(-y_i F_m(\mathbf{x}_i)) = \exp(-y_i F_{m-1}(\mathbf{x}_i)) \exp(-\alpha_m y_i h_m(\mathbf{x}_i))$, soit

$$w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i h_m(\mathbf{x}_i)).$$

Les exemples **mal classés** ($y_i h_m = -1$) voient leur poids **multiplié** par $e^{\alpha_m} > 1$; les bien classés par $e^{-\alpha_m} < 1$. L'itération suivante se concentre donc sur les cas restés difficiles.

5.5 Interprétation biais-variance

AdaBoost réduit principalement le **biais** : chaque itération affine la frontière de décision et rend le modèle plus expressif. Revers : sur des données **bruitées**, l'accent excessif mis sur des points aberrants (impossibles à bien classer) peut augmenter la variance et sur-apprendre.

6 Gradient Boosting

6.1 Motivation

Le Gradient Boosting généralise AdaBoost à une **fonction de perte arbitraire** $L(y, F)$ (régression, classification, etc.). On minimise le risque empirique

$$R(F) = \sum_{i=1}^n L(y_i, F(\mathbf{x}_i)).$$

6.2 Descente de gradient dans l'espace des fonctions

Idee centrale : traiter les valeurs $F(\mathbf{x}_i)$ comme des « paramètres » et faire une descente de gradient sur eux. La direction de plus forte décroissance est l'opposé du gradient. À l'itération m , on calcule les **pseudo-résidus** :

$$r_i^{(m)} = - \left. \frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right|_{F=F_{m-1}}. \quad (2)$$

Exemple 6.1 (Cas de la perte quadratique). Pour $L(y, F) = \frac{1}{2}(y - F)^2$, on a $-\frac{\partial L}{\partial F} = (y - F)$. Le pseudo-résidu est alors **exactement le résidu ordinaire** $r_i = y_i - F_{m-1}(\mathbf{x}_i)$. « Booster » revient à ajuster successivement les arbres sur les erreurs restantes : c'est l'intuition la plus simple du boosting.

6.3 Rôle des arbres et mise à jour

Les pseudo-résidus $\{r_i^{(m)}\}$ ne dépendent que des données actuelles. On entraîne un **arbre de régression** h_m pour les approximer : $\mathbf{x}_i \mapsto r_i^{(m)}$. On cherche ensuite le pas optimal (recherche linéaire) :

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(\mathbf{x}_i) + \gamma h_m(\mathbf{x}_i)),$$

puis on met à jour avec shrinkage :

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu \gamma_m h_m(\mathbf{x}).$$

6.4 Lien avec AdaBoost

AdaBoost est le **cas particulier** du Gradient Boosting pour la perte exponentielle $L(y, F) = \exp(-yF)$. Dans ce cas, les pseudo-résidus sont proportionnels aux poids w_i d'AdaBoost : les deux algorithmes coïncident.

6.5 Interprétation biais–variance

Le Gradient Boosting réduit principalement le **biais** (chaque itération corrige une part d'erreur systématique ; les arbres sont volontairement peu profonds). Le shrinkage ν et le nombre d'itérations M contrôlent la variance et le sur-apprentissage.

7 XGBoost

7.1 Objectif régularisé

XGBoost améliore le Gradient Boosting par une **régularisation explicite** et une optimisation du second ordre. Il minimise

$$\mathcal{L} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_t \Omega(f_t), \quad \Omega(f) = \gamma K + \frac{\lambda}{2} \sum_{j=1}^K w_j^2,$$

où Ω pénalise la complexité de chaque arbre : K est le nombre de **feuilles**, w_j le poids de la feuille j , γ pénalise le nombre de feuilles et λ la magnitude des poids.

7.2 Approximation de Taylor d'ordre 2

À l'itération t , on ajoute f_t à la prédiction courante $\hat{y}_i^{(t-1)}$. On développe la perte à l'ordre 2 :

$$\ell(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) \approx \ell(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2,$$

avec le **gradient** et le **hessien** de la perte par rapport à la prédiction :

$$g_i = \frac{\partial \ell}{\partial \hat{y}_i}, \quad h_i = \frac{\partial^2 \ell}{\partial \hat{y}_i^2}.$$

En écartant les termes constants (indépendants de f_t), l'objectif à l'itération t devient

$$\tilde{\mathcal{L}}^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \gamma K + \frac{\lambda}{2} \sum_{j=1}^K w_j^2.$$

7.3 Poids optimal d'une feuille (dérivation)

Un arbre envoie chaque exemple dans une feuille ; soit I_j l'ensemble des exemples tombant dans la feuille j , qui prédit la valeur constante w_j . On regroupe la somme par feuille et on pose

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i.$$

L'objectif s'écrit alors comme une somme de paraboles en w_j :

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^K \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma K.$$

Chaque terme est minimisé en annulant sa dérivée : $G_j + (H_j + \lambda) w_j = 0$, d'où le **poids optimal** :

$$w_j^* = -\frac{G_j}{H_j + \lambda} \quad (3)$$

En réinjectant w_j^* dans la parabole (un terme $G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2$ vaut $-\frac{1}{2} \frac{G_j^2}{H_j + \lambda}$ à l'optimum), on obtient le **score de structure** de l'arbre :

$$\tilde{\mathcal{L}}_{\min}^{(t)} = -\frac{1}{2} \sum_{j=1}^K \frac{G_j^2}{H_j + \lambda} + \gamma K. \quad (4)$$

Plus ce score est bas, meilleure est la structure de l'arbre.

7.4 Gain d'une coupure (*split*)

Pour décider de couper une feuille en deux (gauche L , droite R), on compare le score avant/après à l'aide de (4). Le **gain** est

$$\text{Gain} = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right) - \gamma. \quad (5)$$

On ne réalise la coupure que si le gain est positif. Le terme $-\gamma$ agit comme un **seuil** : une coupure qui n'améliore pas assez le score est rejetée (élagage automatique).

Sur quoi travaille XGBoost

XGBoost n'utilise pas directement les résidus, mais le **gradient** g_i (erreur locale, « dans quelle direction corriger ») et le **hessien** h_i (courbure locale, « avec quelle confiance »). Les arbres regroupent les exemples à gradients similaires pour une correction ciblée, et la régularisation λ, γ évite les arbres trop complexes.

7.5 Interprétation biais–variance

XGBoost agit sur les **deux** composantes : réduction du *biais* par l'apprentissage itératif (comme tout boosting), et contrôle de la *variance* par la régularisation (λ, γ), le sous-échantillonnage des lignes/colonnes, et le shrinkage ν .

8 Synthèse biais–variance

Méthode	Structure	Cible principale	Mécanisme clé
Bagging	Parallèle	Réduire la variance	Bootstrap + moyenne
Random Forest	Parallèle	Réduire la variance	+ décorrélation (variables aléatoires)
AdaBoost	Séquentiel	Réduire le biais	Re-pondération, perte exponentielle
Gradient Boosting	Séquentiel	Réduire le biais	Pseudo-résidus (gradient fonctionnel)
XGBoost	Séquentiel	Biais <i>et</i> variance	Taylor d'ordre 2 + régularisation

À retenir

- **Bagging / Random Forest** : on moyenne des modèles indépendants et instables $\Rightarrow$ *moins de variance*. Le Random Forest décorrèle en plus les arbres pour briser le plancher $\rho\sigma^2$.
- **Boosting** : on ajoute séquentiellement des modèles qui corrigent les erreurs $\Rightarrow$ *moins de biais*. AdaBoost = re-pondération ; Gradient Boosting = ajustement des pseudo-résidus.
- **XGBoost** : boosting + information du second ordre (gradient, hessien) + régularisation explicite $\Rightarrow$ contrôle fin du compromis biais/variance.